資料結構：契約、表示與成本

資料結構不只是儲存資料的形狀；每個操作都要回答：接受什麼、回傳什麼、成功後狀態如何改變，以及代價如何隨輸入成長。本章以可追蹤的狀態與操作說明這四件事。

演算法與程式

演算法對合法輸入給出明確步驟，並須在有限步後完成；持續運作的服務程式不一定終止。本章把終止性視為操作契約的一部分。輸入可以為空，但操作必須說明空輸入的行為。

先寫抽象值，再談存法

抽象資料型態（ADT）定義可觀察的值與操作；表示（representation）是如何在記憶體中保存它。兩個實作的儲存配置可以不同，但若對每個合法操作產生相同可觀察結果，便實現相同規格。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

有容量 stack 的契約

固定容量 $C \geq 0$，抽象值是長度不超過 C 的有限序列 S，左端為底、右端為頂。create(C) 得到空序列；isEmpty 和 isFull 分別判斷長度為 0、C。push(x) 在未滿時回傳 success 並得到 $S \parallel [x]$；pop 在非空時回傳尾元素並刪除它。滿時 push 回傳 failure，空時 pop 回報錯誤，均不改變 S。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

相同值可以重複

stack 是有序序列，不是只記元素的集合：$[2, 2]$ 經一次 pop 仍是 $[2]$。圖中以三個槽位對照實際儲存與抽象序列。

容量 3：從陣列表示得到有序序列



α(a, size) = [a[0], …, a[size − 1]]

斜線槽位不屬於抽象值；相同元素仍佔不同位置。

表示不變量與抽象函數

對陣列 a、容量 C 和整數 s，合法狀態要求 $0 \le s \le C$，且 $a[0], \ldots, a[s-1]$ 均為有效元素。抽象函數 $\alpha(a, s) = [a[0], \ldots, a[s-1]]$。槽位 $a[s..C)$ 的殘留值不在抽象序列中；不可讀 $a[C]$。

逐步證明 push 保持契約

假設表示不變量成立且 $s < C$。先成功寫入 $a[s] = x$，再令 $s \leftarrow s + 1$。新長度仍不超過 C，舊前綴不變，故新抽象值是 $\alpha(a, s) \,\|\, [x]$。若已滿，先回報錯誤、不寫槽位，抽象值維持原狀。

pop 與失敗狀態

若 $s > 0$，取出 $a[s-1]$ 後令 $s \leftarrow s - 1$；新抽象序列恰好少掉原尾端。若 $s = 0$，不能讀 $a[-1]$；應先回報錯誤。殘留在舊槽位的數值不代表仍在 stack 裡。若 $s > C$，表示本身已非法，抽象函數不適用。

C++ 的值與例外

回傳值可用 `std::optional<T>` 表示空 pop，也可由契約指定例外；兩者必須讓呼叫者知道失敗。若複製或移動 `T`、配置記憶體可能丟出例外，不能先改 `size` 再假設操作成功。只有在所選型別與操作滿足相應不丟例外或可回滾條件時，才承諾失敗後狀態不變。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

搜尋的規格不同，答案也不同

本章用 0-based、非遞減陣列與 lower_bound：回傳第一個不小於 x 的位置 $i \in [0, n]$；若所有元素都小於 x，回傳 n。演算法科 FIRST-MATCH 用 1-based 索引，找不到回 NIL。lower_bound 的 n 是合法插入位置，不等同找不到就回 NIL。

lower_bound 的完整不變量

對自然非遞減順序，每輪維持 $0 \le l \le r \le n$；所有 $0 \le j < l$ 有 $A[j] < x$，所有 $r \le j < n$ 有 $A[j] \ge x$。未分類元素是 $[l, r)$；候選插入位置為閉區間 $[l, r]$。重複鍵必須返回第一個不小於的位置。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

lower_bound：可執行的更新規則

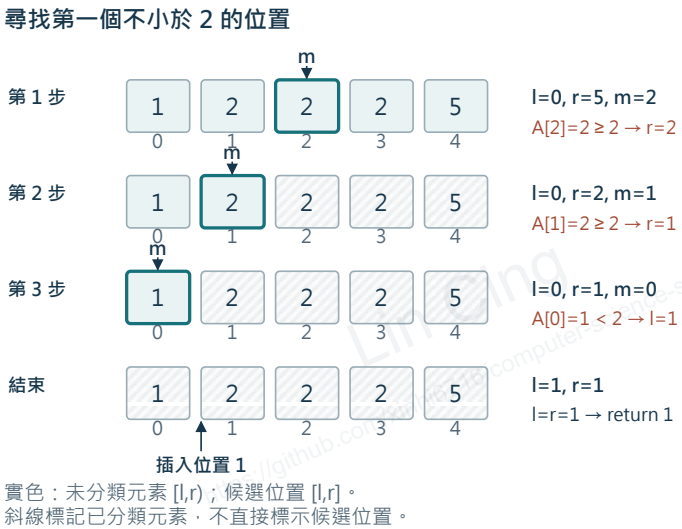
輸入 $A[0..n)$ 已由小到大排列；回傳第一個不小於 x 的位置。右端點不包含在搜尋區間內。

```
l ← 0;  r ← n
while l < r:
    m ← l + floor((r - l) / 2)
    if A[m] < x:  l ← m + 1
    else:         r ← m
return l
```

初始化時兩側區間皆空；每次更新只排除已確認的一側，並保留所有可能的插入位置。即使全部元素都小於 x，回傳 n 也不會讀取 $A[n]$。

一次搜尋的三輪追蹤

取 $A = [1, 2, 2, 2, 5]$、$x = 2$。每次比較後，不變量仍成立；最後 $l = r = 1$，因此回 1。空陣列立即回 0；$x = 6$ 回 5，不會讀取 $A[5]$。



實色：未分類元素 [l,r)；候選位置 [l,r]。
斜線標記已分類元素，不直接標示候選位置。

搜尋不變量：初始化與兩種更新

令 k 是第一個不小於 x 的位置，若不存在則 $k = n$。證明每輪開始時 $k \in [l, r]$。

- **初始化：** $l = 0, r = n$，因此 $k \in [0, n]$。兩側已分類區間均為空。
- **若 $A[m] < x$：** 排序性保證 $A[0..m]$ 全都小於 x，所以 $k \geq m + 1$。設 $l = m + 1$，仍保留 k。
- **若 $A[m] \geq x$：** 排序性保證 $A[m..n)$ 全都不小於 x，所以 $k \leq m$。設 $r = m$，仍保留 k。
- **終止：** 當 $l = r$，唯一的候選位置就是 $k = l$。即使 $k = n$ 或資料含重複鍵，推理仍成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為什麼搜尋會停止？

當 $l < r$，令 $m = l + \lfloor (r - l)/2 \rfloor$，故 $l \leq m < r \leq n$，讀取 $A[m]$ 合法。若 $A[m] < x$，將 $[l, m]$ 的元素歸入左側；否則將 $[m, r)$ 的元素歸入右側，位置 m 仍可能是答案。整數 $r - l$ 每輪嚴格下降；停止時 $l = r$，兩側已分類區間推出後置條件。

比較器與可執行語意

C++ 比較器 comp 須符合 strict weak ordering，陣列依同一比較器排序。一般形式中，左區皆滿足 comp(A[j],x)，右區皆不滿足；結果是第一個使 comp(A[i],x) 為 false 的位置。先檢查 $i < n$，才能讀取 A[i]；再以 !comp(A[i],x) 與 !comp(x,A[i]) 同時成立確認等價，不能直接用 == 代替。使用 `m=l+(r-l)/2` 需確保索引型別能表示 n，且差與加法不溢位。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

比較次數不是全部時間

在隨機存取、每次比較 $O(1)$ 的模型下，lower_bound 最壞使用 $\Theta(\log(n+1))$ 次比較。若每個字串最多 L 個字元，一次字典序比較最多做 $O(L)$ 次字元操作；總字元操作為 $O(L\log(n+1))$。含尋址與控制等固定成本，總時間上界是 $O((L+1)(1+\log(n+1)))$。空陣列無元素比較，仍有常數控制成本。

成本先寫清楚

固定空間、輸入儲存、輸出與額外工作空間可以分開報告。時間可按比較、元素搬移或位元操作計算。C++ `vector` 的配置、建構與搬移成本取決於元素型別與 allocator；用單位成本模型時要先聲明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

加法次數與位元成本

依序把 n 個整數加到累加器。若每次加法的**兩個運算元與結果**都能用至多 L 位表示，採逐位加法時，每次成本為 $O(L)$，總成本為 $O(nL)$；$n=0$ 時僅有初始化成本。

只知道最後結果很短並不夠。例如 $2^N+(-2^N)=0$，結果只需常數位，但兩個輸入都需 $\Theta(N)$ 位；光讀取完整輸入便可能需要 $\Omega(N)$ 時間。

若使用固定寬度機器字，且題目保證不溢位，才可在 word-RAM 模型把一次加法計為 $O(1)$，此時總時間為 $O(n)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

遞迴與活躍框架

對非負整數 n，$fac(0) = 1$，$fac(n) = n\,fac(n-1)$。求 $fac(3)$ 依序呼叫 3、2、1、0，共 4 次呼叫、3 條呼叫邊；最深同時有 4 個活躍框架。回傳順序得到 1、1、2、6。這裡把每次呼叫算一次，並不把大整數乘法當免費。

Fibonacci 的兩種基底

若 $F(0) = 0, F(1) = 1$，則 $F(5) = 5$；若改成 $G(0) = G(1) = 1$ 且使用相同遞迴，則 $G(5) = 8$。直接遞迴若把初次呼叫也算入，$C(0) = C(1) = 1$、$C(n) = 1 + C(n-1) + C(n-2)$，故 $C(5) = 15$、$C(8) = 67$；不要將呼叫次數與框架最大深度混為一談。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

乘以 2 不等於呼叫兩次

採來源層級逐次呼叫的單位成本模型，$n = 2^k \geq 1$，基底 $T(1) = \Theta(1)$。一個呼叫 `2*Test(n/2)+8` 的時間為 $T(n/2) + \Theta(1) = \Theta(\log n + 1)$；兩個獨立呼叫相加則為 $2T(n/2) + \Theta(1) = \Theta(n)$。不假設編譯器做共同子式消除。

同樣乘以 2，不代表多呼叫一次

2 × Test(n/2) + 8

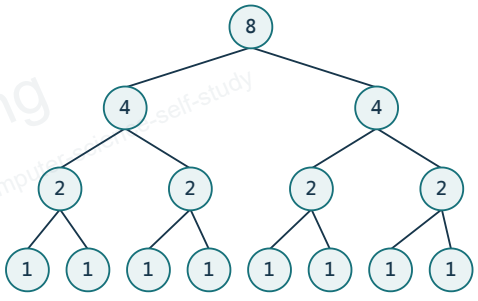
每層只計算一次



含基底共 4 次呼叫；時間 Θ(log n)

Test(n/2) + Test(n/2) + 8

每層分成兩次呼叫



含基底共 15 次呼叫；時間 Θ(n)

模型：程式層級計數，未消除或共用重複呼叫；n 從 8 開始，基底為 n ≤ 1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

迴圈結束前還有一次測試

固定 $n \geq 1$，對 $i = 1, \ldots, n$ 設 $x = n$，只要 $x \geq 0$ 就做 $x \leftarrow x - i$。內迴圈本體精確執行 $\lfloor n/i \rfloor + 1$ 次；每個 i 還多一次失敗的條件測試。$n = 6$ 時本體次數依序為 7、4、3、2、2、2，合計 20；含最後失敗的測試共 26。

從精確計數推到成長率

總本體次數 $S(n) = \sum_{i=1}^{n} (\lfloor n/i \rfloor + 1)$。因 $n/i - 1 < \lfloor n/i \rfloor \leq n/i$，得到 $nH_n < S(n) \leq nH_n + n$。以積分界 $\log(n+1) \leq H_n \leq 1 + \log n$，因此 $S(n) = \Theta(n \log n)$。此式假設 $n \geq 2$，空範圍另處理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

大 O、下界與成本函數

$f \in O(g)$ 意味存在常數 $c > 0, n_0$，使所有 $n \geq n_0$ 有 $0 \leq f(n) \leq cg(n)$；Ω 反向界定，Θ 要上下界都成立。最壞成本是 $W(n) = \max_{|x|=n} T(x)$；O 是界的關係，不等同「最壞情況」。

最壞、期望與攤銷

最壞成本取所有合法輸入的最大值；期望成本須指定輸入或演算法隨機性分布；攤銷成本則對一串操作求總成本，再分配到各次操作。三者可同時出現在同一演算法，不能互相取代。

重複鍵會改變 quicksort

若二路 Lomuto 分割把等值鍵都放到 pivot 同側，全相等輸入每輪為 $n - 1$ 與 0；即使 pivot 均勻隨機，成本仍是 $\Theta(n^2)$。相異鍵且隨機 pivot 的標準模型才有期望 $\Theta(n \log n)$；三路分割會把小於、等於、大於鍵分開，等值區不再遞迴。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

動態陣列的真實搬移

初始容量 $c_0 = 3$，滿時把容量加倍；連續 append 13 個元素時，在第 4、7、13 次分別搬 3、6、12 個，總共 21 個。把任意初始容量誤寫成 $1 + 2 + 4 + 8 = 15$ 會低估本例。



為何一串 append 是線性？

設初始容量 $c_0 \geq 1$。$n = 0$ 時搬移與寫入均為 0；以下假設 $n \geq 1$。若未擴容，搬移為 0；否則搬移容量為 $c_0, 2c_0, \ldots, 2^k c_0$，最後一次舊容量 $c = 2^k c_0 < n$。等比級數總和 $2c - c_0 < 2n$。再加 n 次新元素寫入，總元素寫入小於 $3n$。此處不計初始容量配置成本；若算配置，須依 allocator 與元素建構模型另加。

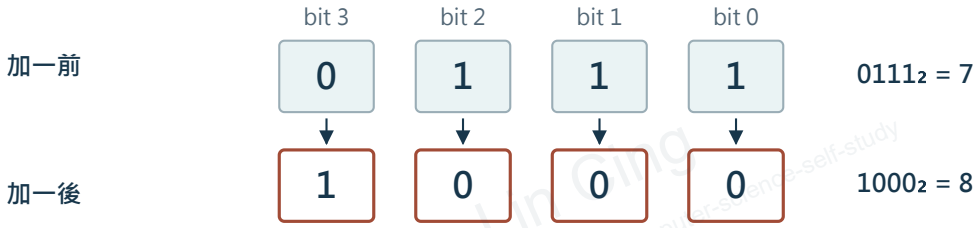
勢能法：讓便宜操作先存下信用

對只做 append 的倍增表，取 $\Phi(s, C) = \max(0, 2s - C)$，初始 $s = 0$ 故勢能為 0。未擴容時勢能增加至多 2，攤銷成本至多 $1 + 2 = 3$。滿表容量 $C > 0$ 時，擴容成本 $C + 1$、勢能由 C 變 2，攤銷成本為 3。勢能非負，故任意前綴的實際總成本不超過攤銷總和。

位元計數器的勢能例子

固定 4 bit、以模 16 計數；每翻轉一位算 1 次寫入，取勢能為 1 的個數。$0111 \to 1000$ 寫 4 位，勢能由 3 降至 1，攤銷成本為 $4 - 2 = 2$。一般翻轉 t 個尾端 1 並設定一個 0，勢能變化 $1 - t$，攤銷成本仍為 2。全 1 回捲全 0 時寫 4 位、勢能減 4。

二進位計數器加一：實際成本與攤銷成本



4 個 bit 翻轉：實際成本 c = 4；勢能 Φ = 1 的個數。

Φ前 = 3，Φ後 = 1；攤銷成本 ĉ = 4 + (1 − 3) = 2。

此例假設位數足夠，忽略計數器整體配置成本。

結束前檢查四件事

規格是否寫出空輸入、重複值與失敗行為？實作狀態是否有可驗證的不變量和抽象函數？證明是否同時處理保持與終止？成本是否明確說明操作、輸入大小和期望或攤銷的適用條件？下一章再把這些工具用在實際的線性結構。

參考資料與署名

Lin Cing，《資料結構》手寫筆記。

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study